

KeyQuorum Lab

Browser GUI Manual

Bailey Forbes

A guide to KeyQuorum Lab's own browser interface: its header and tabs, and the guided, gated tutorials built into the page. For the `keyquorum` / `keyquorum-device` command-line tool the lab runs underneath, see the separate KeyQuorum manual.

Contents

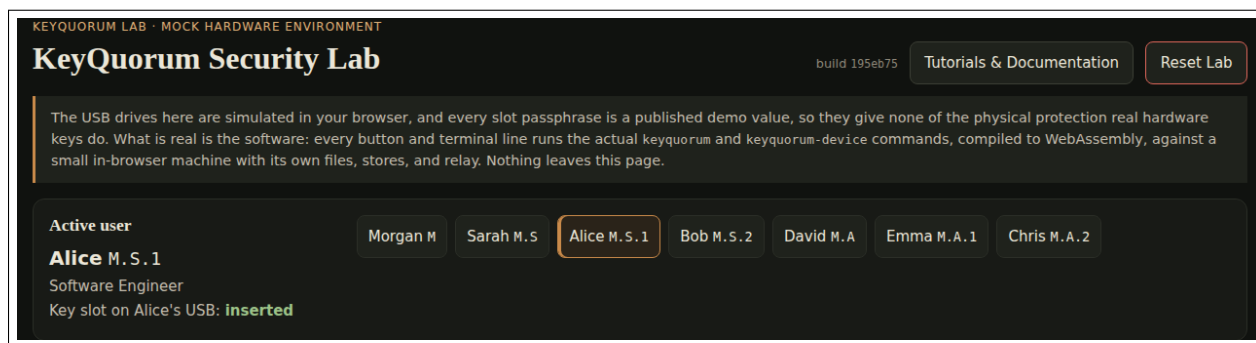
1	Overview	1
1.1	Default credentials	1
1.2	Identity and drives	2
2	The tabs	2
2.1	Organization key tree	2
2.2	Mock USB devices	3
2.3	File Explorer	4
2.4	Inbox and sent	4
2.5	Security & devices	4
2.6	Activity and access trace	5
2.7	Terminal	6
3	Guided tutorials	6
3.1	How a step works	7
3.2	The modules	8
3.2.1	Identities & drives	8
3.2.2	Files & unlocking	8
3.2.3	Mailbox: sending & receiving	8
3.2.4	File history	8
4	Where the lab lives	9

1 Overview

KeyQuorum Lab runs the KeyQuorum crate in a browser as a small sandboxed machine, compiled to WebAssembly. It has its own files, a SQLite store per person, mock USB drives, and a mailbox relay answering in the page. Every button and every terminal line runs the real `keyquorum` and `keyquorum-device` commands — there is no separate “demo mode” standing in for them.

Note

This manual documents the lab’s own GUI: its header, tabs, and guided tutorials. It does not re-document `keyquorum` / `keyquorum-device` themselves — that is the separate *KeyQuorum* manual, which the lab runs underneath its interface unchanged. Where a control here is a thin wrapper over a CLI concept (custody, quorum, private bridges, authenticated updates, the mailbox relay), this manual names that concept without re-explaining it.



The lab header: build commit, the Tutorials & Documentation and Reset Lab buttons, the disclaimer, and the Active user bar.

Reset Lab discards everything the session has done — every drive move, sent letter, provisioned key, password file, export, share link, signature, and activity-log entry — and replaces it with a fresh copy of the seeded demo state, instantly. Nothing here is persisted outside the browser tab, so this is the only way back to a known starting point; it is also what **Exit tutorial** and the tutorial picker leave alone; a gated tutorial in progress does not survive a reset.

The lab itself is a browser-based virtual machine: its mock USB drives emulate USB-token custody entirely in memory, and every slot passphrase is a published default value (Section 1.1), so they give none of the physical protection real hardware keys do. What is real is the software underneath: every action, in every tab, is one of the same commands documented in the KeyQuorum manual.

1.1 Default credentials

Every credential the lab uses is a published placeholder, not a secret. The lab ships as a public WebAssembly bundle that runs entirely in your browser, so anything baked into it — the seeded people, their USB passphrases, and the seeded files — is visible to anyone who inspects the bundle. That is by design: none of it guards real data, so nothing here needs to stay confidential, and no real file or hardware key is ever at risk of passing through the lab.

USB slot passphrases. Every seeded USB slot unlocks with `lab-demo-<label>`, where `<label>` is that slot’s dotted tree label:

Person	Label	Passphrase
Morgan	M	lab-demo-M
Sarah	M.S	lab-demo-M.S
Alice	M.S.1	lab-demo-M.S.1
Bob	M.S.2	lab-demo-M.S.2
David	M.A	lab-demo-M.A
Emma	M.A.1	lab-demo-M.A.1
Chris	M.A.2	lab-demo-M.A.2

Terminal default values. Typing a command yourself at the Terminal tab — rather than clicking a GUI action

that stages your own value first — is answered with the same published defaults a real prompt would use, whenever it asks for a password or PIN:

- Password: `lab-demo-password`
- PIN: `0000`

Note

Seeded quorum-protected files never use a password. They are protected by KeyQuorum’s quorum and device-custody policies instead, and are unlocked with the USB identities above.

1.2 Identity and drives

The **Active user** bar shows who you are acting as right now, and which mock USB drive their personal key slot lives on. Clicking another name’s chip switches to them, the way sitting down at a different desk would. Most actions need that person’s drive *inserted* first — a mock drive behaves like a real one: its slots are only usable while it is connected, and moving a slot onto someone else’s drive needs both drives inserted at once, matching the physical requirement of moving a token between two real USB drives.

2 The tabs

At desktop width every tab’s panel is visible at once, stacked on one scrolling page; the tab row itself only appears at phone width, where it switches which single panel is shown. What follows is every panel, in the order they appear on the page.

2.1 Organization key tree

The org-wide split tree, drawn as the dotted-label hierarchy it is: `M`, `M.S`, `M.S.1`, and so on. You only ever see your own slice — your lineage, your descendants, siblings, and established-bridge peers. A leaf tagged *required* contributed a share the last time you unlocked a file; *satisfied* means that share was actually presented.

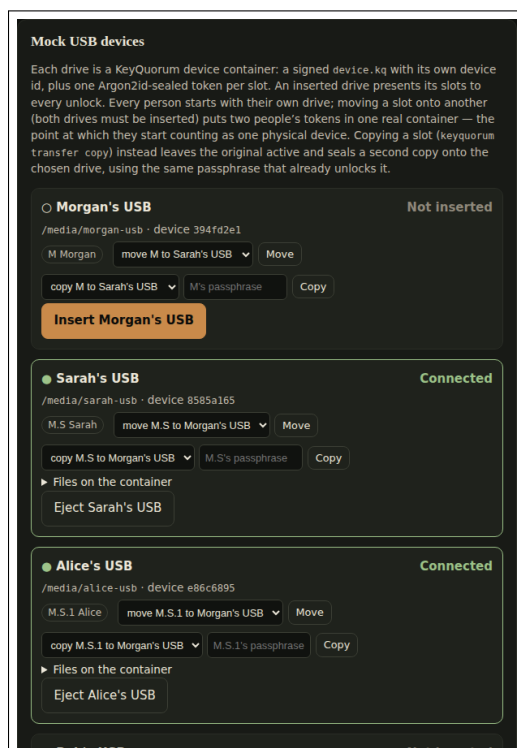
- **Revoke key** bans a leaf’s hardware key outright; it drops that leaf’s bindings and pairings but does not evict it or refresh survivor shares.
- **Reissue...** replaces a leaf’s hardware key onto an already-provisioned replacement token.
- **Bridges** whitelists and establishes cross-branch links, the same two-step allow-then-add process the CLI uses.
- **Tree restructure** walks through proposing a new public generation and collecting the parent’s countersignature when one is required.



The organization key tree, showing the dotted-label hierarchy, per-leaf slot status, and the Revoke/Reissue controls.

2.2 Mock USB devices

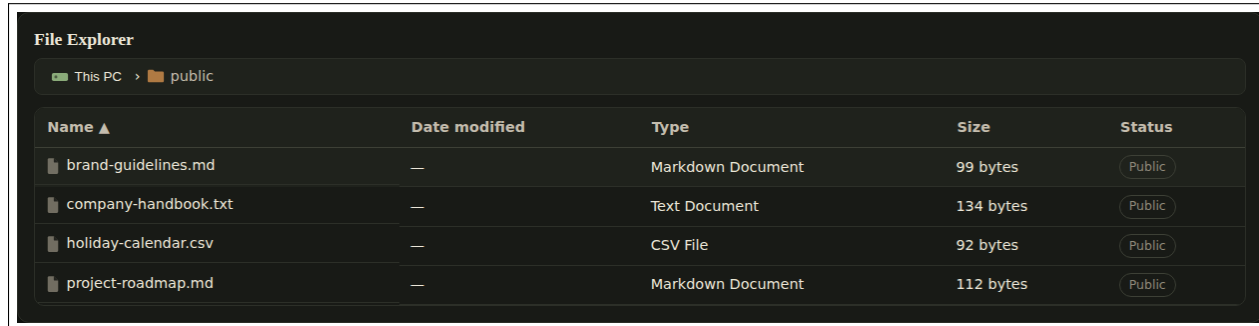
Each card is one device container: a signed device descriptor plus one sealed token per slot. **Insert** / **Eject** plug a drive in or pull it out. **Move** asks for a destination drive (both drives must be inserted) and relocates a slot's keypair onto it — the point where two people's tokens land in one physical container and start counting as a single device toward any multi-device quorum. **Copy** instead leaves the original slot active and seals a second copy elsewhere, using the same passphrase that already unlocks it — a seeded slot's own published default passphrase (Section 1.1).



Mock USB devices: connected state, the slots each drive carries, and the Move / Copy controls.

2.3 File Explorer

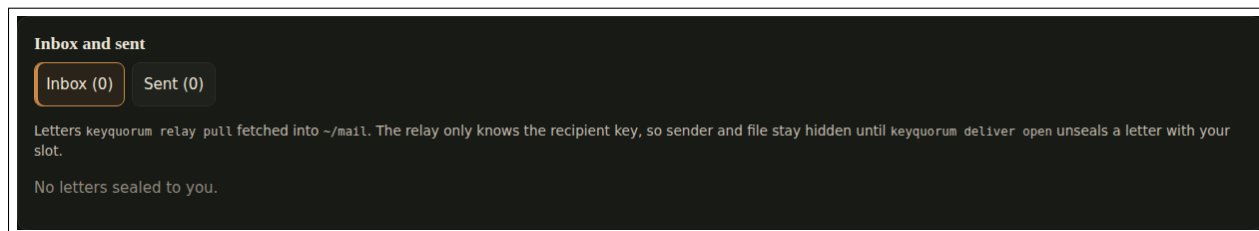
A Windows-Explorer-style view onto the active user's files, grouped into folders. Double-clicking a file (or selecting it and pressing **Open**) unlocks and views it — a quorum file prompts for the shares it needs, a password file for its password. **Send...** seals a copy to another label's key and drops it in their inbox; **Sign** is only offered on public or received plaintext; **Properties** shows the file's protection, size, and timestamps. Right-clicking a row offers the same actions. A file with an expiry stamp shows a status pill that flips to *expired* on its own while the tab is open.



The File Explorer, showing the public folder's contents and their status pills.

2.4 Inbox and sent

Letters are sealed to a recipient's key and routed by the mailbox relay without it ever reading them — sender and contents stay hidden until **Receive** unseals a letter with your own slot. **Reject** answers with a signed refusal instead. The Sent view tracks whether a delivery is still awaiting the recipient's signed acknowledgement, and **Check relay for acknowledgements** pulls any that are waiting.



Inbox and sent, in the empty seeded state.

2.5 Security & devices

Everything that is not the hardware-key quorum tree or a file: locking a note with a password (and optionally a PIN), exporting a file or credential as a portable sealed bundle, minting a time-limited share link, provisioning a fresh slot on an inserted drive and registering it as a new leaf, signing or verifying with a private sign bridge you belong to, and, further down, read-only device logs and relay counters for the in-process mailbox this lab runs against.

Security & devices

Password-locked files

Protect a note with a password only you choose (keyquorum access password), with an optional 4-digit PIN.

File name

my-note.txt

Contents

Whatever you want protected with a password

Your own lock password

☐ Also require a 4-digit PIN

Lock with my password

Your password-locked files

None yet.

Share links

Time-limited, revocable links (keyquorum share create-file). A bearer token authorizes redemption, not identity — anyone given the token can use it below.

None yet.

Exported bundles

Portable KQXB bundles you sealed to another lab user's public key (keyquorum export file). No import step exists yet, so this only shows the sealed bytes.

None yet.

Create a new key

Provision a fresh slot on an inserted drive, sealed under a passphrase you choose.

Drive

Sarah's USB

New slot label

e.g., M.F

Your own passphrase

Security & devices: the password-lock form, exported bundles, share links, and key/leaf provisioning.

2.6 Activity and access trace

Every action taken leaves a trace here: which checks ran, in order, and whether each one passed. The **Equivalent operation** line under the latest trace is the literal command line that action ran — the same one that could be typed in the Terminal tab. The full log below keeps every action for the session, not just the last one.

Tracked files. The Activity tab is also where tracked files live. The **Tracked files** panel acts as whoever is active, with their own store and their own drive's slot; each button runs one real `keyquorum file` command:

- **Track and sign** a new file, then **Check in** edits — signed with your slot, or unsigned to see a revision stay *pending*.
- Per revision: **Sign revision** (its author), **Approve as parent** (the author's direct parent countersigning), **Finalize** (the scope owner or an ancestor, once it is trusted), **View** an older revision, and **Diff** it against its parent. Each revision shows its user label first, when it has one, then its generated label and short id.
- **Rename file**: change the name shown; the file id and every revision id stay the same.
- **Share file** with another person. The line above the button says which revision would go — the head, or the last trusted revision when the head is pending. The recipient **Accepts** or **Refuses** the letter (into their own copy), and the sender **Records the answer**.
- **Ask for a file, or for a change**: pick a person, choose *a file* (send me your copy) or *a change to this file*, add a message if you like, and press **Send request** (`keyquorum file request`, with `-change` for a change). Under *Requests for a file or a change* the holder sees the message and presses **Accept request** or **Decline request** (`open-request`, then `answer-request`, which also records the request in their own copy of the file). The asker presses **Record the request answer** (`open-answer`). A request only asks: nothing is delivered or changed. To serve an accepted file request the holder uses **Share file**; a change arrives as an ordinary revision. The log shows the request id, kind and decision, never the message.
- When two copies diverge: **Import copy**, **Review the fork** (each side's changed lines and the named reviewer) and **Merge heads**; a clean merge is a new revision that still has to be signed. **Resolve** runs `keyquorum file resolve` as the active person with their own slot: keep the left or right side, use text you type, or reject a proposed merge. Only the assigned reviewer (or an ancestor of theirs) can settle a conflict, so the same button is refused for anyone else; the lab adds no check of its own.
- **Verify history**, **Export history snapshot** and **Check** a snapshot against the file.

- **Schedule expiry** or **Destroy content now**: the file becomes a tombstone that still verifies.
- **Link gate**: record unlocks of a quorum file, or of one of your password files, in this file's history; **Unlink** stops it.

Whatever a button or a Terminal command writes to a tracked file appears in the log at once, right below the action that caused it. History entries can be expanded (revision, labels, parents, trust, history root) and filtered by **Tracked File**, **Revision**, **Security**, **Sharing** and **Conflict**; a revision graph per file sits under the log. Three tracked files are seeded: `budget.txt` (an unsigned edit, so sharing falls back), `forecast.txt` (two edits that merged on their own) and `memo.txt` (two edits to one line, assigned to a reviewer).

Tracked files

Acting as Sarah (M.S). Each button runs a real `keyquorum file` command with your slot; what it records appears in the log below.

New tracked file

notes.txt

First revision's text

Track and sign

Tracked file `budget.txt` — `/srv/keyquorum/tracked/budget.txt.kqtf`

budget.txt scope M.S · 6 events · root 93c42812 · `/srv/keyquorum/tracked/budget.txt.kqtf`

0999c6be Q4 baseline by M.S (root) TRUSTED View

0ecb2d76 Late edit, unsigned by M.S ← 0999c6be · head PENDING MissingContentSignature View Diff Sign revision

The head is not trusted, so sharing sends 0999c6be, the last trusted revision.

Verify history

Export history snapshot

Record unlocks of `architecture.md (quorum)` Link gate

Edit the current revision

Q4 budget: 125000

Short description (optional)

☒ Sign this edit with my slot

Check in

Expires (UTC) `mm/dd/yyyy, --:-- --` Schedule expiry Destroy content now

Share with `Morgan (M)` Share file

The Activity tab's tracked files panel: revisions with their trust, what sharing would send, and the controls for the selected file.

2.7 Terminal

A real shell on the same lab machine. Every button on every other tab runs a command here under the hood; typing it directly works identically, including commands no button exposes yet. Use full paths in `keyquorum` arguments: the lab's shell does not expand `~` inside them, so `-db ~/keyquorum.sqlite` would open a new, empty store rather than your own.

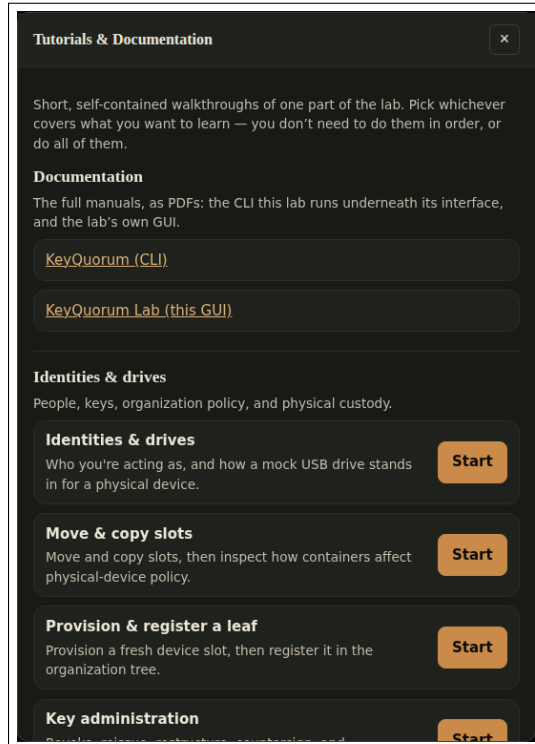
```
keyquorum-device list /media/alice-usb
bridge list 1
usb insert david
su david
```

`usb insert` / `usb eject` and `su` are lab-only shell verbs for drive and identity control; every other line is an ordinary `keyquorum` / `keyquorum-device` command. Any passphrase, password, or PIN prompt a typed command raises is answered with its published default value (Section 1.1), the same way a GUI action's prompt is when you have not staged your own answer first.

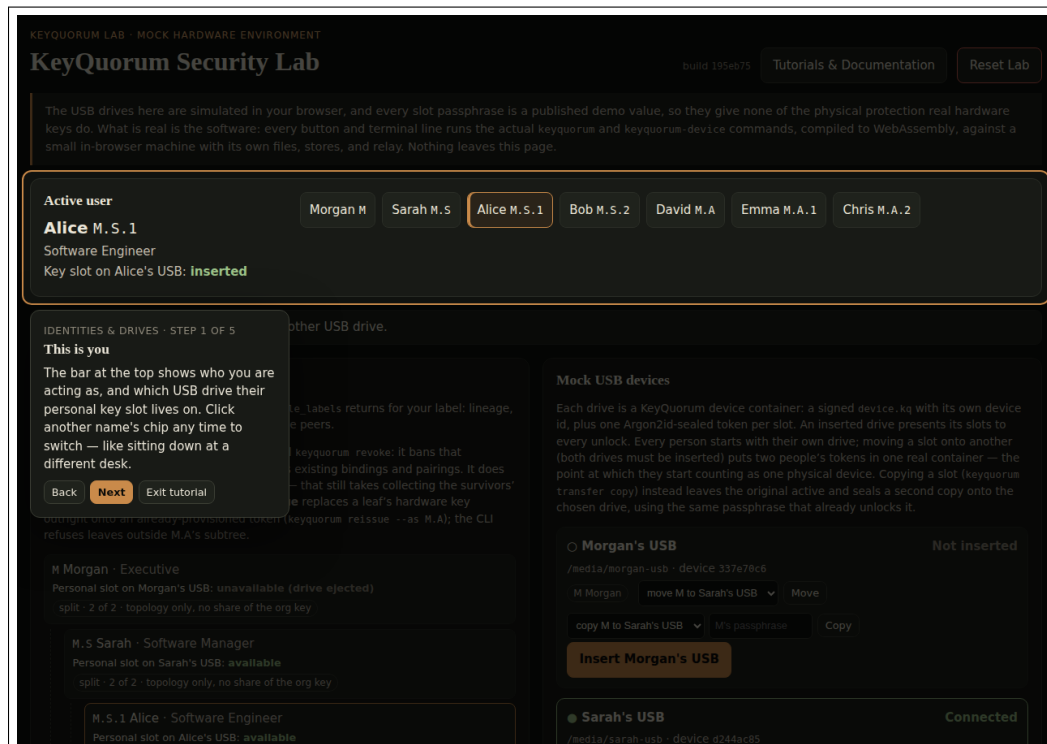
3 Guided tutorials

The **Tutorials & Documentation** button in the header opens a picker of short, self-contained modules, plus a **Documentation** section linking the same two PDF manuals this crate ships everywhere else — the CLI manual and this one. Each module covers one part of the lab; picking one is not a commitment to the others, and they can be

taken in any order.



The picker: its Documentation section (both PDF manuals) above the Identities & drives category.



A tutorial step in progress: the Active user bar is spotlighted, and the tooltip names the step and what to do.

3.1 How a step works

Each step spotlights the part of the page it is about and shows a tooltip with an explanation. A step comes in one of two kinds:

- An **informational** step just explains something; a **Next** button advances it.
- A **gated** step (marked “Waiting for you to try it...”) names a real action to perform — insert a drive, open a file, switch identity, send a letter — and does not advance until that action actually happens in the lab. Completion is checked against the lab’s own state (the current snapshot, or the most recent entry in the Activity log), not against a click on the tooltip itself, so there is no way to fake having done it. Once satisfied, the step marks itself done and advances on its own a moment later. **Skip this step** is always available if you would rather move on without doing it.

A step can also relocate its target as the page changes under it — for example, tracking a folder tile in the File Explorer until it is opened, then tracking the file row that takes its place — so one step can walk you through a short sequence rather than only ever pointing at one fixed element.

3.2 The modules

The picker groups its thirteen modules into three categories; taking one module is never a commitment to the others in its category, or to the category itself.

3.2.1 Identities & drives

- **Identities & drives** — who you’re acting as, bringing a mock USB drive online, reading the organization key tree’s own slice, and switching identity.
- **Move & copy slots** — move and copy slots, then inspect how containers affect physical-device policy.
- **Provision & register a leaf** — provision a fresh device slot, then register it in the organization tree.
- **Key administration** — revoke, reissue, restructure, countersign, and understand parent approval.
- **Bridges & remote devices** — manage cross-branch visibility and distinguish local device actions from relay transfer.

3.2.2 Files & unlocking

- **Files & unlocking** — opening a public file, then trying a quorum-protected one to see what the access trace looks like when a single identity does not meet its threshold.
- **Complete a quorum unlock** — read a file’s requirements, connect cooperating devices, and complete an unlock.
- **Passwords & PINs** — lock a note with your own password and an optional PIN, unlock it again to see the lab’s second, independent protection scheme end to end, then lock a second file straight from the Terminal and watch a typed command fall back to the published default password and PIN (Section 1.1) instead of one you chose.
- **Export bundles & share links** — contrast recipient-sealed export bundles with bearer share links.
- **Signatures, properties & expiry** — sign and verify through a bridge, inspect properties, and observe destructive expiry.

3.2.3 Mailbox: sending & receiving

- **Mailbox: sending & receiving** — sealing a file to another label, switching to that identity, and receiving it from the inbox.
- **Reject & acknowledge** — exercise the rejection path and explicitly refresh a sender’s acknowledgement state.
- **Updates & device letters** — see which higher-level workflows use sealed letters and where their controls actually live.

3.2.4 File history

Four short parts on the Activity tab’s **Tracked files** panel. Each one sets up who is active and which drives are in, so they can be taken in any order.

- **1 · Track, sign & fall back** — as Sarah, track a new file, check in an unsigned edit, see sharing fall back to the last trusted revision, sign the edit, and read the result in the activity log’s Revision filter.
- **2 · Hand a file over** — share `budget.txt` with Alice, switch to her and accept the letter into her own copy, then switch back and record her signed answer in Sarah’s history.

- **3 · Merges & conflicts** — sign the merge of `forecast.txt` that happened on its own, then review `memo.txt`, whose same-line edits could not merge, and see the reviewer its history names.
- **4 · Gate links & expiry** — as Alice, track a file, link `architecture.md`'s quorum gate to it, open that file and find the unlock in the Security filter, then destroy the tracked file's content and verify the tombstone.
- **5 · Ask for a file or a change** — as Sarah, ask Alice for a change to `budget.txt` with a message, switch to her and accept the request, then switch back and record her signed answer; the log shows the request and its answer, never the message.

The step card is docked in the top-right corner of the window (bottom edge on a phone), so it never covers the part of the page a step is about; the target is scrolled into view and outlined. **Move** puts the card on the other side and **Hide text** shrinks it to its title.

Finishing a module's last step opens a short completion notice; from there, or from **Exit tutorial** at any point, the **Tutorials & Documentation** button reopens the picker for another module.

4 Where the lab lives

KeyQuorum Lab is published from `lab/` to GitHub Pages and embedded by `bailey-forbes.com`. A GitHub Actions workflow tests, builds, and publishes it from `main` on every change. The lab is a sandboxed machine, not a second implementation of KeyQuorum: it must never reimplement quorum, custody, approval, visibility, bridge, or delivery rules, or add a gate of its own in front of them — if the CLI would allow something on a real machine, the lab allows it too.